City Brain Convergence — Solution Proposal

1. Executive Summary

City Brain Convergence tasks three independently-built teams with delivering one layer each of a smart-city system — a **Twin** (simulated city), a **City Brain** (decision engine), and a **Guardian** (trust/security watchdog) — that must interoperate as a single live system despite never having seen each other's code. Because our layer assignment is not yet known, this proposal presents our design, technology choices, and security approach for **all three layers**, so that whichever one we are assigned, we can begin implementation immediately with a proven design already in hand.

Our guiding principle across every design decision in this document is the same one the challenge's own scoring structure rewards most heavily: **the shared contract matters more than any single layer's internal cleverness.** We are optimizing for a system that a partner we have never met can integrate against correctly on the first try, and that survives adversarial scenarios it has never seen before. Sections 3–5 detail our design for each layer; Section 6 defines the shared technical contract; Section 7 justifies our technology choices; Section 8 covers our security and trust architecture; Section 9 lays out our implementation plan; Section 10 risk management.

2. Challenge Understanding

The system is split into three layers that must be merged and demonstrated live:

Layer	Role
Twin	Simulates the city — sensors, actuators, and the environment itself
City Brain	Reads sensor data, decides the correct operational response, dispatches actuators
Guardian	Watches the same data stream and asks "is this true?" — detects spoofing/attacks and can veto or contain

Every layer must work correctly against *any* implementation of the other two layers, as long as that implementation honors the shared contract. The system must also keep functioning correctly when the sensor data it depends on is being deliberately falsified.

Interoperability and correct response to falsified/adversarial data are, by the challenge's own scoring structure, weighted more heavily than any individual layer's raw feature count. Our design decisions throughout this proposal are made against that reality: a technically impressive layer that a partner cannot integrate with, or that cannot survive an attack it wasn't specifically coded for, is a weaker solution than a simpler layer built to a clean, well-documented, defensively-designed contract.

---

3. Layer Design: Twin

3.1 Responsibilities

The Twin simulates five sensor types (traffic cameras, water-level sensors, power-load meters, air-quality monitors, emergency-call feeds) and five actuator types (police/ambulance dispatch, traffic-signal control, water-valve control, power-grid switching, device isolation/quarantine), exposing every reading and action as a callable **MCP-style tool**.

3.2 Design Approach

Rather than modeling full sensor physics from scratch, we treat each sensor as a **pluggable signal generator** behind a common interface:

```
generate(sensor_id) -> reading
inject_anomaly(sensor_id, mode)   # spike | freeze/replay | drift | drop-to-zero
```

Each generator produces a baseline pattern (flat, daily sine wave, or Poisson event process depending on sensor type) plus Gaussian noise. This gives us a fully working, controllable Twin quickly, so that City Brain and Guardian integration is never blocked waiting on the simulation core to mature.

The `inject_anomaly` hook is not cosmetic: it is how we test our own Guardian design during development, and it is the same kind of mechanism the organizers' adversarial testing will exercise against us from the outside. Building it in from day one, rather than retrofitting it later, is a deliberate risk-reduction choice.

3.3 Domain Simulator Integration

If our team is assigned the Twin layer, we commit to backing three of the five sensor domains with real physics-based simulators rather than synthetic noise generators, because these are the domains where a toy random-number generator is most visibly "fake" to graders and where real actuator-to-sensor coupling (City Brain closes a valve → the Twin's own physics shows the level actually drop) is the strongest demonstration of a genuinely working system:

- **Traffic (cameras + traffic-signal control):** SUMO, driven via its TraCI Python API, for real vehicle-flow and congestion modeling. A signal change dispatched by City Brain produces genuine downstream congestion effects in SUMO's traffic model, not a scripted number change.
- **Water (water-level sensors + valve control):** WNTR (Water Network Tool for Resilience, Python wrapper around EPANET) for real hydraulic simulation across a defined pipe/tank network. Closing a valve actually changes downstream pressure and tank levels.
- **Power (power-load meters + grid switching):** pandapower for real power-flow simulation across a defined bus/line topology. Switching a grid segment actually changes downstream load/voltage readings.

Air-quality monitors and emergency-call feeds remain lightweight synthetic generators (seasonal curve + noise, and a Poisson arrival process respectively) — no equivalent open, low-setup simulator exists for these two domains, and their realism does not depend on physics-level modeling the way traffic/water/power do.

Phasing, so this does not put the core system at risk:

1. **First implementation pass:** every sensor, including traffic/water/power, ships behind the lightweight generator interface from Section 3.2, with the `inject_anomaly` hook in place. This guarantees a fully working, MCP-exposed Twin exists before any simulator integration begins — the contract and cross-layer integration path are never blocked on simulator setup.
2. **Second pass:** define the SUMO road network, WNTR pipe/tank topology, and pandapower bus/line topology in parallel (one team member per simulator), each behind the *same* MCP tool signatures already frozen in Section 6 — City Brain and Guardian never see the swap.
3. **Validation pass:** all three simulators are integrated and swapped in behind the existing interface, validated against our own City Brain/Guardian in isolation before being exposed to any partner.
4. **Fallback rule:** if any single simulator is not passing its own integration tests, that domain reverts to its Section 3.2 generator, with integration resumed later. The interface is identical either way, so this fallback is a config change, not a rewrite — the committed plan does not risk our interoperability even if one simulator runs behind schedule.

3.4 Actuators

Actuators are modeled as state-mutating tool calls (e.g. `set_traffic_signal(intersection_id, state)`) that update the Twin's internal state and are reflected in subsequent sensor readings from affected sensors — this is what makes the "City Brain dispatches → Twin's readings change" loop real rather than cosmetic, and is directly demoable during integration with partner teams.

4. Layer Design: City Brain

4.1 Responsibilities

City Brain consumes the Twin's live sensor stream, detects real-world incidents (accidents, fires, traffic jams, flooding), decides the correct operational response, and dispatches the matching actuator — but only after checking Guardian's trust verdict on the triggering reading.

4.2 Incident Detection

Each incident type is detected from a distinct, explicit pattern over the raw sensor stream rather than a single-reading threshold, so that one noisy sample cannot trigger a false dispatch:

Incident	Detection pattern
Flooding	Water-level reading exceeds a zone threshold and stays above it for N consecutive readings (sustained, not a single spike)
Fire	Correlated signal across air-quality (smoke/particulate spike) and, where available, emergency-call feed for the same zone within a short time window
Traffic accident	Emergency-call feed reporting an incident in a zone, corroborated by a sudden drop in traffic-camera flow/speed for that same zone
Traffic jam	Sustained low average speed / high vehicle density from traffic cameras over a rolling window, distinguishing a jam from normal rush-hour variance

Requiring sustained or corroborated signals (rather than a single reading) is itself a first line of defense against spoofed single-reading attacks, complementing — but not replacing — Guardian's dedicated trust verdict.

4.3 Decision Engine

We use an **explicit decision table / rule-based engine** rather than a black-box model, covering all four incident types:

```
IF incident_type == "flooding"        AND guardian_trust(reading) >= THRESHOLD
  THEN dispatch("water_valve_control", target=affected_zone)

IF incident_type == "fire"             AND guardian_trust(reading) >= THRESHOLD
  THEN dispatch("emergency_dispatch", unit="fire", target=affected_zone)

IF incident_type == "traffic_accident" AND guardian_trust(reading) >= THRESHOLD
  THEN dispatch("emergency_dispatch", unit="ambulance", target=affected_zone)

IF incident_type == "traffic_jam"      AND guardian_trust(reading) >= THRESHOLD
  THEN dispatch("traffic_signal_control", target=affected_intersection)
```

This is a deliberate choice, not a shortcut: decision logic here is a fixed enum of incident → actuator mappings, and a rule table is trivially explainable during live demo Q&A — an advantage when judges are actively probing "why did your system do that?" A model's prediction is much harder to defend on the spot. If the ruleset grows past roughly 15 conditions (e.g. severity tiers per incident type), we will migrate to a lightweight rules-engine library (e.g. `json-logic`) rather than a nested if/else tree, purely for maintainability.

4.4 Guardian Gating

Every dispatch decision first calls Guardian's trust-verdict endpoint for the triggering reading(s). If Guardian flags a reading as untrusted, City Brain suppresses the action and logs the suppression with the reason code Guardian returned — this is both a correctness requirement (City Brain must respect Guardian's verdict before acting) and a direct demonstration of our security posture.

4.5 Incident Lifecycle and Idempotency

City Brain tracks each detected incident as a stateful object (`open` → `dispatched` → `resolved`) keyed by zone and incident type, rather than re-evaluating raw readings independently on every tick. This prevents the same sustained flood or jam from re-triggering duplicate ambulance/valve/signal dispatches on every subsequent reading, and gives us a clean audit trail of "what did City Brain decide, and when" for both debugging during integration and for live demonstration.

4.6 Extensibility

New incident types are added by defining a new detection pattern (Section 4.2) and a new row in the decision table (Section 4.3) — no change to the dispatch mechanism, Guardian-gating logic, or lifecycle tracking is required, which keeps the core engine stable while incident coverage grows.

---

5. Layer Design: Guardian

5.1 Responsibilities

Guardian watches the same sensor stream as City Brain and independently evaluates whether each reading is trustworthy, detecting sensor malfunction, actuator failure, spoofed data, and cyberattacks — and can isolate a sensor, quarantine a device, or roll back a falsified reading.

5.2 Design Approach — Hybrid Detection

We use a **two-layer hybrid** rather than a single detection method, with explicit rules mapped to each named threat:

Threat	Detection mechanism
Replay attack	Rule: reject a reading whose timestamp/nonce has already been seen
Spoofed / out-of-range data	Rule: reject a reading outside the sensor's physically possible range
Tampered message	Rule: reject a reading with a missing or invalid HMAC signature
Sensor malfunction / drift	Statistical: z-score / EWMA outlier detection against that sensor's own recent history
Actuator failure	Rule: actuator's reported post-command state doesn't match the commanded state within a timeout
Novel/unclassified anomaly	Statistical layer catches anything the rule layer wasn't written for

We deliberately **do not** use a trained ML model (e.g. Isolation Forest) as our primary detector. Adversarial scenarios in the later stages of the challenge are organizer-provided and unseen in advance — a model trained on our own Twin's synthetic data risks overfitting to our own generator's quirks rather than generalizing to the actual attack patterns we'll be tested against. Rules plus statistics are more defensible against genuinely novel scenarios and are far easier to explain and debug live.

5.3 Trust Verdict Interface

Guardian exposes a `get_trust_verdict(reading_id)` tool returning a verdict (`trusted` / `suspect` / `rejected`) plus a reason code drawn from the table in Section 5.2, which City Brain

must query before acting — this is the core interoperability contract between the two decision-making layers and is specified precisely in Section 6.

5.4 Containment and Rollback

On a rejected verdict, Guardian takes one of three concrete actions rather than only logging the rejection:

- **Isolate:** call the Twin's `isolate_sensor` tool so the compromised sensor's readings are excluded from City Brain's stream until cleared.
- **Quarantine:** call the Twin's `quarantine_device` tool for a suspected compromised actuator, blocking further commands to it.
- **Rollback:** Guardian maintains a short rolling cache of each sensor's last N *trusted* values; on detecting a falsified reading, it can supply the last known-good value back to City Brain in place of the rejected one, so a legitimate incident response isn't stalled by a single bad reading sitting in an otherwise-healthy stream.

All three actions are demonstrated live, not just logged, since containment behavior is a directly observable part of the system's security story during a demo.

5.5 Guardian Self-Resilience

Because Guardian is itself a target — an attacker who can fool Guardian defeats the whole system's trust model — we avoid relying on any single detection signal. The rule layer and statistical layer are independent and run in parallel; a verdict requires agreement logic between them (e.g. either layer alone can reject a reading, but both must agree for `trusted`), so an attack crafted to slip past one detector alone does not automatically earn a `trusted` verdict.

5.6 Extensibility

New attack signatures are added as new rule entries in Section 5.2's table without touching the statistical layer, and the statistical layer's thresholds are configured per sensor type rather than hardcoded, so tuning against a new partner's Twin (with different sensor value ranges) is a configuration change, not a code change.

---

6. Shared Technical Contract

Because our layer partners are assigned to us rather than chosen, our single highest-leverage engineering artifact is a **frozen, versioned API contract**, committed to the repository's `contract/` directory from the very start of development — before we know who our partners are.

Contract contents:

- **OpenAPI 3.x specification** for all REST/tool endpoints (readings, actuator calls, trust-verdict queries)
- **JSON Schema** definitions for every message type: sensor reading (`sensor_id`, `type`, `value`, `unit`, `timestamp`, `signature`, `nonce`), actuator command, trust verdict
- **MCP tool manifest** listing every callable tool with its input/output schema, generated directly from our FastMCP tool definitions so the manifest can never drift out of sync with the running code
- A short **integration guide** (plain-English, not just schema) describing how to authenticate, subscribe to the live stream, and interpret a trust verdict — written for a partner who has never seen our code

This contract is versioned in git and treated as append-only once a partner has begun integrating against it: breaking changes are not made mid-integration.

---

7. Technology Stack and Justification

Every choice below is optimized for two things our solution prioritizes above all else: a contract an unfamiliar partner can integrate against on the first try, and a system that survives unseen adversarial scenarios — over raw performance or technical novelty that a 4-person team cannot afford to gold-plate.

Decision	Choice	Why
Language/framework	Python 3.11 + FastAPI	Async-native for streaming; auto-generates OpenAPI docs (contract documentation for free); one language covers City Brain's rule logic and Guardian's statistics without a stack switch
Real-time transport	WebSockets	Lowest integration cost for an unfamiliar partner — one client line in any language, no broker dependency to stand up correctly under time pressure
MCP tool exposure	FastMCP	Full MCP compliance (explicitly required for Twin) with the least boilerplate for a team new to the protocol
Shared contract format	JSON Schema + OpenAPI	Language-agnostic, human-readable, auto-generated from FastAPI, skimmable by a partner in a browser
Containerization	Docker + docker-compose	Guarantees `docker compose up` works for a partner without the operational overhead Kubernetes would demand from a 4-person team
Message integrity	HMAC signing + timestamp/nonce	Directly defends against the named threat model (spoofed/replayed readings) with no PKI/certificate coordination needed between unfamiliar partners
Sensor simulation (Twin)	Custom lightweight generator first, then SUMO (traffic), WNTR (water), pandapower (power) integrated behind the	Working, controllable Twin quickly so the contract is never blocked; real physics-based simulators for the three

Decision	Choice	Why
	same MCP interface; air-quality/emergency-calls stay generator-based	domains where actuator-sensor coupling is most visibly demoable, with a same-interface fallback if a simulator runs behind
Decision logic (City Brain)	Rule-based decision table	Fully explainable live during demo Q&A; a fixed enum of incident types doesn't need a trained model
Anomaly detection (Guardian)	Hybrid: rules + statistical outlier detection	Robust to unseen adversarial scenarios; avoids overfitting risk of a model trained only on our own synthetic Twin data

---

8. Security and Trust Architecture

Security is a core requirement of our solution, not an add-on layered in afterward.

Message-level defenses (all layers):

- Every sensor reading and actuator command is HMAC-signed with a pre-shared secret established at integration setup with a partner
- Every message carries a timestamp and single-use nonce; receivers reject stale or reused nonces to defeat replay attacks
- Schema validation is enforced at every service boundary (reject malformed messages before they reach decision logic)

Guardian-specific:

- Hybrid rule + statistical detection (Section 5.2) covering both known attack signatures and unseen statistical anomalies
- Independent-agreement verdict logic (Section 5.5) so no single detection method is a single point of failure
- Every trust verdict includes a machine-readable reason code, so City Brain's suppression behavior — and our own debugging during integration — is auditable, not a black box
- Containment and rollback actions (Section 5.4) are logged with full context for post-incident review

Cross-layer principle: No layer trusts another layer's *output* implicitly — City Brain never acts on a reading without a fresh Guardian verdict, and Guardian's own containment calls to the Twin go through the same signed, schema-validated interface as everything else. This "trust nothing by default, verify every message" posture is our single core security design principle and is repeated consistently across all three layer designs above.

---

9. Implementation Plan

We structure our build around technical milestones rather than a fixed calendar, so the plan holds regardless of which layer we are assigned:

1. **Contract-first foundation.** Freeze the OpenAPI spec, JSON Schemas, and MCP tool manifest (Section 6) before writing feature logic. Every subsequent milestone builds against this frozen interface.
2. **Core layer skeleton.** Bring up a minimal working version of whichever layer we are assigned — lightweight Twin generators (Section 3.2), a basic City Brain rule table (Section 4.3), or Guardian's rule-layer checks (Section 5.2) — validated in isolation against the frozen contract using tools like Postman/curl/swaggger, with no dependency yet on a real partner.
3. **Depth pass.** Layer in the harder parts specific to our assignment: domain simulator integration if Twin (Section 3.3), incident lifecycle tracking if City Brain (Section 4.5), or the statistical detection layer and containment/rollback logic if Guardian (Sections 5.2–5.4).
4. **Integration validation.** Test our layer against a self-built mock of the other two layers, exercising both the happy path and adversarial inputs (via our own `inject_anomaly` hook), before ever connecting to a real partner.
5. **Cross-team integration.** Connect to actual partner implementations, using the integration guide (Section 6) to resolve any ambiguity, and iterate on our contract documentation based on real friction points encountered.
6. **Hardening.** Expand Guardian's rule coverage and City Brain's incident patterns based on integration lessons learned, in preparation for adversarial testing scenarios we have not seen in advance.

At every milestone, a fallback to the simpler version of any component (e.g. generator instead of physics simulator, as in Section 3.3) is available with no interface change, so schedule pressure on any one sub-component never blocks the rest of the system.

---

10. Risk Management

Key risks and mitigations:

Risk	Mitigation
Partner's implementation diverges from the shared contract	Contract is schema-validated and versioned; integration guide (Section 6) written for a stranger, reducing ambiguity that causes divergence
Guardian overfits to our own Twin's synthetic data and fails unseen adversarial scenarios	Hybrid rule + statistical detection (Section 5.2), deliberately avoiding a model trained solely on our own generator
Domain simulator (SUMO/WNTR/pandapower) topology setup overruns schedule (if Twin)	Lightweight generator ships first behind the same MCP interface (Section 3.3); same-interface fallback reverts any late simulator to its generator with no rework
Integration bugs surface only once a real partner connects	Dedicated cross-team integration validation milestone (Section 9, step 4) using our own mock of the other layers before a real partner is available

---

11. Conclusion

Our proposal treats City Brain Convergence's real technical challenge as what it actually is: not "build an impressive layer," but "build a layer that provably interoperates with strangers and survives an attack you haven't seen." Every architectural decision in this document — the frozen OpenAPI/JSON-Schema contract, the HMAC-signed message security model, the hybrid rather than pure-ML Guardian, the explicit incident-detection patterns in City Brain, and the milestone-based implementation plan with built-in fallbacks — traces back to that priority. We are ready to begin implementation immediately upon confirmation of our layer assignment.
